

# LOOPS

## Overview

Python has two main loop types:

- `while` loops
    - Used when the number of repetitions is not known ahead of time
  - `for` loops
    - Used when looping through a container or a known range of values
- 

## Topic: While Loop vs. For Loop

### While Loop

Used when the number of iterations is not easily known before the loop starts.

```
i = 0
while i < 100:
    # loop body statements
    i += 1
```

### For Loop

Used when the number of iterations is known or when iterating through a container.

```
for i in range(100):
    # loop body statements
```

## Main Differences

| While Loop                                        | For Loop                                                    |
|---------------------------------------------------|-------------------------------------------------------------|
| Must initialize the loop variable before the loop | Usually handles iteration automatically                     |
| Must manually update the loop variable            | Less likely to become an infinite loop                      |
| Good when the stopping point is unknown           | Good when the number of iterations is known                 |
| Useful for input-based repetition                 | Useful for containers like lists, strings, and dictionaries |

## Topic: While Loops

### Syntax

```
while expression:
    # loop body

# statements after loop ends
```

### Key Vocabulary

- Sentinel value
  - A value that causes a loop to stop
- Infinite loop
  - A loop that never ends because its condition stays `True`

### Notes

- Make sure the loop condition will eventually become `False`
- Update loop variables correctly to avoid infinite loops

## Topic: For Loops

## Syntax

```
for variable in container:
    # loop body

# statements after loop ends
```

## Notes

- The loop variable stores each item from the container one at a time
- Works with lists, strings, dictionaries, tuples, ranges, and more

## Example: Looping Through a List

```
fruits_list = ["apple", "banana", "cherry"]

for fruit in fruits_list:
    print(fruit)
```

## Output

```
apple
banana
cherry
```

## Breakdown

- `fruit` = variable holding each element
- `fruits_list` = the container being looped through

## Example: Searching in a List

```
my_list = [5, 6, 7]

for x in my_list:
    if x == 5:
        print("Found element 5")
```

## Output

```
Found element 5
```

## Example: Looping Through a String

```
my_str = "Yes"

for char in my_str:
    print(char)
```

## Output

```
Y
e
s
```

## Example: Looping Through a Dictionary

```
channels = {
    "MTV": 35,
    "CNN": 28,
    "NBC": 4
}
```

```
for channel_name in channels:
    print(f"{channel_name} is on channel {channels[channel_name]}")
```

## Output

```
MTV is on channel 35
CNN is on channel 28
NBC is on channel 4
```

## Example: Printing a List in Reverse

```
names = ["Armando", "Lena", "Aidan", "Norah"]

# normal order
for name in names:
    print(name, "|", end=" ")

# reverse order
for name in reversed(names):
    print(name, "|", end=" ")
```

## Output

```
Armando | Lena | Aidan | Norah |
Norah | Aidan | Lena | Armando |
```

## Notes

- `reversed()` prints items in reverse order
- `end=" "` keeps output on the same line with spaces instead of new lines

## Topic: `range()` Function

### Notes

- `range()` creates a sequence of integers
- It is commonly used in `for` loops
- A range has:
  - start
  - stop
  - step
- `range` objects are immutable

### Common `range()` Examples

| Range                        | Generated Sequence     | Explanation                      |
|------------------------------|------------------------|----------------------------------|
| <code>range(5)</code>        | <code>0 1 2 3 4</code> | Starts at 0 and stops before 5   |
| <code>range(0, 5)</code>     | <code>0 1 2 3 4</code> | Starts at 0 and stops before 5   |
| <code>range(3, 7)</code>     | <code>3 4 5 6</code>   | Starts at 3 and stops before 7   |
| <code>range(10, 13)</code>   | <code>10 11 12</code>  | Starts at 10 and stops before 13 |
| <code>range(0, 5, 1)</code>  | <code>0 1 2 3 4</code> | Counts by 1                      |
| <code>range(0, 5, 2)</code>  | <code>0 2 4</code>     | Counts by 2                      |
| <code>range(5, 0, -1)</code> | <code>5 4 3 2 1</code> | Counts backward by 1             |
| <code>range(5, 0, -2)</code> | <code>5 3 1</code>     | Counts backward by 2             |

## Topic: Nested Loops

### Definition

A nested loop is a loop inside another loop.

## Notes

- Total iterations = outer loop iterations × inner loop iterations
  - Outer loop usually controls rows
  - Inner loop usually controls columns
- 

## Nested While Loops Example

```
row = 1
while row <= 3:
    col = 1
    while col <= 4:
        print(f"{row},{col}", end=" ")
        col += 1
    print()
    row += 1
```

## Output

```
1,1 1,2 1,3 1,4
2,1 2,2 2,3 2,4
3,1 3,2 3,3 3,4
```

---

## Nested For Loops Example

```
for row in range(1, 4):      # 1 to 3
    for col in range(1, 5):  # 1 to 4
        print(f"{row},{col}", end=" ")
    print()
```

## Output

```
1,1 1,2 1,3 1,4
2,1 2,2 2,3 2,4
3,1 3,2 3,3 3,4
```

## Topic: **break** and **continue**

### **break**

#### Definition

**break** immediately exits the loop you are currently in.

#### Example

```
nums = [2, 4, 7, 8]

for n in nums:
    if n == 7:
        break
    print(n)
```

#### Output

```
2
4
```

### **continue**

#### Definition

**continue** skips the rest of the current iteration and moves to the next one.

#### Example

```
nums = [2, 4, 7, 8]
```

```
for n in nums:  
    if n == 7:  
        continue  
    print(n)
```

## Output

```
2  
4  
8
```

## Topic: Loop **else**

### Definition

A loop can have an **else** block that runs only if the loop ends normally and does not hit a **break**.

### While Loop with **else**

```
while expression:  
    # loop body  
else:  
    # runs if loop ends normally
```

### For Loop with **else**

```
for item in iterable:  
    # loop body
```

```
else:
    # runs if loop ends normally
```

## Important

- The `else` block does not run if `break` is used

## Topic: Getting Both Index and Value While Looping

Sometimes you need both:

- the position of an item
- the value of that item

Example list:

```
origins = [4, 8, 10]
```

### Method 1: Using `range()` and `len()`

```
origins = [4, 8, 10]

for index in range(len(origins)):
    value = origins[index]
    print(f"Element {index}: {value}")
```

## Output

```
Element 0: 4
Element 1: 8
Element 2: 10
```

## Method 2: Using `list.index()`

```
for value in origins:
    index = origins.index(value)
    print(f"Element {index}: {value}")
```

### Note

This works, but it is usually not the best choice if values repeat in the list.

---

## Method 3: Using `enumerate()`

```
for index, value in enumerate(origins):
    print(f"Element {index}: {value}")
```

### Output

```
Element 0: 4
Element 1: 8
Element 2: 10
```

### Why `enumerate()` is Best

- Cleaner
  - Easier to read
  - Gives both index and value at the same time
- 

## Quick Summary

### Use a `while` loop when:

- You do not know exactly how many times the loop should run
- The loop depends on user input or a changing condition

## Use a `for` loop when:

- You know the number of iterations
- You are looping through a container like a list, string, or dictionary

## Important loop tools

- `range()` for counting
- `reversed()` for reverse order
- `break` to exit early
- `continue` to skip one iteration
- `enumerate()` to get both index and value